CampusFlow

An Explainable AI Delivery Robot Simulation

ROHAN SAXENA | 24BCE0801

Abstract

CampusFlow investigates how a simulated campus delivery robot can choose a useful route when the shortest route is not necessarily the cheapest and sensor evidence is uncertain. A browser-based grid environment combines four search methods, a Bayesian blockage estimate, an explicit decision rule and a delivery mission controller. The user can edit the map, compare algorithms on identical inputs, run a delivery and insert an obstacle during execution. The robot then searches again from its current location. All decisions and comparison metrics are visible.

1. Problem and proposed idea

A delivery between a campus depot and a laboratory may encounter crowded walkways, fixed barriers or a temporarily blocked shortcut. A robot that follows a fixed path cannot respond to these changes. A robot that minimizes only the number of moves may also cross expensive crowded areas. The case study therefore asks: how do search strategy, travel-cost modelling and uncertain observations change the route selected by an agent?

The contribution is an integrated, explainable teaching simulation: the same map supports algorithm comparison, belief updates and action-level replanning. The implementation uses classical AI directly and requires no trained model, paid service, API key or physical robot.

Objectives

Compare uninformed and informed search; distinguish move count from weighted travel cost; demonstrate one Bayesian evidence update; represent a delivery using action preconditions; and recover from a newly observed route obstruction.

Agent element	CampusFlow representation
Performance	Complete delivery and return; avoid blocked cells; reduce defined route cost.
Environment	16 × 12 campus grid with walkways, crowds, barriers and a sensor zone.
Actuators	Move up/down/left/right; load, deliver and return.
Sensors	Known map, current location and a simulated binary obstacle report.

Scope: an educational software simulation of one robot and one parcel. The map and probabilities are synthetic; no deployment on a real campus is claimed.

Search model and algorithm design

2. State-space representation

For a single route, a state is the robot location (x, y). The initial state is the depot or current position; the goal is the laboratory or depot, depending on the mission phase. A transition moves to an adjacent, in-bounds, traversable cell. Diagonal moves are excluded. The route state space contains at most 192 locations. The broader mission also stores the current phase and whether the delivery has occurred.

The default depot is at column 2, row 10 and the laboratory at column 15, row 3, using one-based coordinates. The environment is fully specified for each search snapshot. A user-inserted obstacle changes that model during execution; uncertainty is represented only for the designated sensor zone.

Travel cost and heuristic

$c(u, v) = \text{terrain}(v) + 4 \times P(B \mid \text{evidence}) \times I(v \text{ belongs to sensor zone})$
Normal terrain costs 1 unit and crowded terrain costs 6 units. Barriers cannot be entered. The initial location is not charged. The risk term is a chosen expected penalty per zone cell, not a measured energy or time value.

$f(n) = g(n) + h(n)$, with $h(n) = |x - x_{goal}| + |y - y_{goal}|$.
Every legal move costs at least 1, so Manhattan distance is a lower bound and is consistent on this four-neighbour grid. A* can therefore optimize the defined route cost for each fixed snapshot [1].

Method	Expansion rule	What the comparison shows
BFS	FIFO queue; shallowest first	Fewest moves on this graph; unequal costs are ignored.
DFS	LIFO stack; deepest first	A feasible route can be unnecessarily long.
Greedy Best-First	Smallest h(n)	Goal-directed search can miss cheaper alternatives.
A*	Smallest g(n) + h(n)	Balances cost already paid with estimated remaining distance.

Implementation details

The engine stores parent links for route reconstruction, accumulated costs and an explored set. A* retains improvements to previously reached states and ignores outdated frontier entries. The small demonstration uses a sorted array for priority selection rather than a binary heap. Equal priorities follow insertion order. Neighbours are considered right, up, down and left; DFS reverses insertion to preserve this expansion preference.

BFS and DFS serve as uninformed baselines [2]. The page exposes the selected route and explored nodes after a search; robot movement is animated separately. Uniform-cost search is implemented as a test oracle and is not a fifth user-facing comparison option.

Uncertain evidence and delivery planning

3. Bayesian obstacle reasoning

Let B mean that the designated shortcut zone is blocked, and + mean that its simulated sensor reports an obstacle. The four purple-bordered cells share one zone-level belief. They are not four independent Bayesian variables. Illustrative assumptions are P(B) = 0.15, P(+ | B) = 0.85 and P(+ | not B) = 0.10. These values are fixed teaching inputs, not estimates learned from observations.

P(B | +) = [0.85 × 0.15] / [(0.85 × 0.15) + (0.10 × 0.85)] = 0.60.
For a negative report, P(B | -) = [0.15 × 0.15] / [(0.15 × 0.15) + (0.90 × 0.85)] = 0.02857, approximately 2.9%. Without evidence, the model retains the 15% prior. Bayes' rule provides the update from a prior to a posterior [3].

Decision rule: if sensor evidence is enabled and the posterior is at least 0.50, mark the entire zone unavailable. Otherwise keep it traversable and add the expected penalty to each zone-cell entry. Repeatedly toggling a report recomputes from the same prior; the interface does not accumulate repeated evidence or treat toggles as new independent observations.

4. Goal-directed mission controller

Action	Preconditions	Effects
Load	Robot at depot; mission ready	Parcel loaded; outbound travel enabled.
Navigate	A route exists; next cell traversable	Robot advances one grid cell.
Deliver	Robot at laboratory; parcel loaded	Delivery recorded; return phase begins.
Return	Delivery recorded; depot reachable	Robot navigates back; mission completes.
Replan	Next route cell newly blocked	Discard remaining route; search from current position.

The sequence is controlled by a finite-state mission controller. It illustrates action preconditions and a small task decomposition; it is not a general STRIPS, Graphplan or hierarchical task-network solver. Route planning is delegated to the selected search algorithm. The return route is computed separately and may differ from the outward route.

Observation-to-action cycle

Observe map and evidence → update belief → apply blockage rule → search for route → execute next move → observe again. If the user blocks the next route cell, the agent records the event and replans. If no path remains, it halts and reports failure instead of declaring delivery complete.

The implementation uses explicit conditional rules. It demonstrates a narrow link to logic reasoning, without claiming first-order unification, resolution, a full rule engine, or multiagent planning.

Simulation environment and experimental method

5. Simulation architecture

The application contains a browser interface, a pure JavaScript search engine and a mission controller. The interface collects map edits and sensor settings. The engine returns the route, total cost and expansion order. The controller animates movement, checks arrival conditions and requests new search after a disruption. An explanation log records why a route, delivery or replan action occurred. Everything runs locally in the browser.

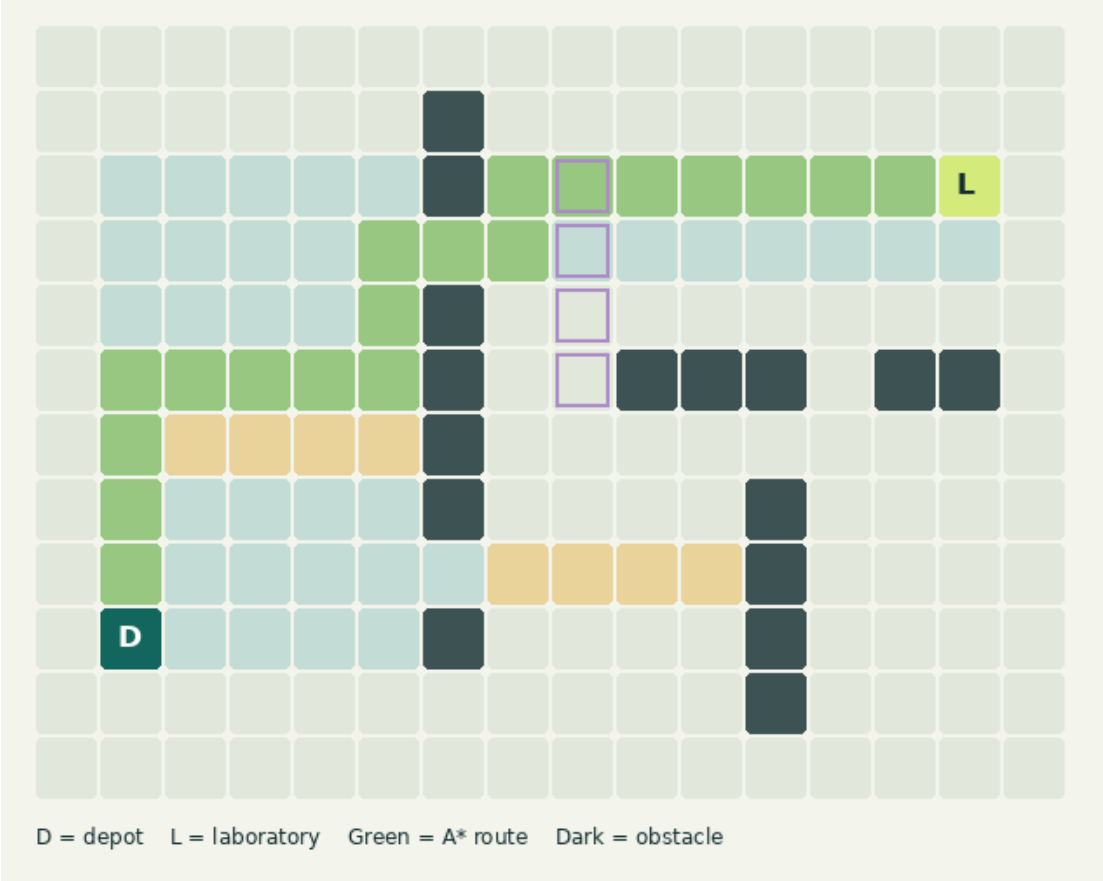


Figure 1. Programmatically rendered state of the default Campus crossing scenario after A* search. The figure uses the same map and search output as the application; it is not a browser screenshot. Pale teal indicates explored cells.

Experimental protocol

Four fixed scenarios are supplied: Open campus, Campus crossing, Crowded laboratory and Disconnected campus. For each comparison, all algorithms use the same initial map, depot, goal, prior and sensor setting. Table 1 on the next page uses sensor evidence disabled, leaving the 15% prior and its expected penalty active. The maps and neighbour order are deterministic.

Measures are route moves, weighted travel cost and nodes expanded, including the goal when found. Failed searches have no move count or path cost. The results describe individual outbound routes; they do not report full-mission cost or real travel time. Animation delay changes display speed only and is excluded from evaluation.

Results and verification

6. Measured search outcomes

Scenario	Algorithm	Moves	Cost	Expanded
Open	BFS	20	20.00	184
Open	DFS	22	22.00	140
Open	Greedy	20	20.00	21
Open	A*	20	20.00	108
Campus	BFS	20	40.00	168
Campus	DFS	22	42.00	116
Campus	Greedy	20	40.00	23
Campus	A*	20	20.60	54
Crowded lab	BFS	20	50.00	168
Crowded lab	DFS	22	47.00	116
Crowded lab	Greedy	20	50.00	23
Crowded lab	A*	26	31.00	170
Disconnected	BFS	No route	-	89
Disconnected	DFS	No route	-	89
Disconnected	Greedy	No route	-	89
Disconnected	A*	No route	-	89

Table 1. Direct engine outputs from the supplied scenarios, with default sensor settings and a risk weight of 4. Costs are simulation units.

On Campus crossing, BFS and A* both use 20 moves, but A* reduces cost from 40.00 to 20.60, a 48.5% reduction. In Crowded laboratory, A* takes 26 moves rather than BFS's 20, while reducing cost from 50.00 to 31.00, a 38% reduction. This demonstrates why minimizing moves and minimizing weighted cost are different objectives.

A* is not always the method with the fewest expansions: Greedy expands 23 nodes in Crowded laboratory, while A* expands 170 to obtain the cheaper route. Every method correctly reports no route for the disconnected map. These are scenario-specific outcomes, not universal performance claims.

Verification evidence

Eleven automated checks passed. They cover Manhattan-optimal BFS on an open grid, legal route reconstruction, A* cost equality with uniform-cost search on all four presets, positive and negative Bayesian updates, obstacle-triggered route replacement, a zero-length route and an isolated start. A further check covers 120 seeded random weighted maps, comparing A* against uniform-cost search and validating every returned path. Test seed: 71.

A deterministic disruption test blocks the first cell of the original A* route, searches again and confirms that the new valid route excludes that cell. These engine checks establish algorithm behaviour; they do not measure hardware reliability, real sensor accuracy or real-world delivery success.

Discussion, applications and conclusion

7. Interpretation and limitations

CampusFlow shows that an agent needs an explicit objective: a short route can be expensive when crowds or uncertainty are represented in the cost. The Bayesian component separates a sensor report from certainty, and the action log makes the consequence of each report inspectable. Replanning connects abstract state-space search with visible behaviour during a delivery.

The grid is small and synthetic. Crowding is a fixed cost, not a moving crowd model. The single sensor zone uses a simplified probability calculation and a manually chosen threshold; no Bayesian belief network or approximate inference is implemented. Replanning reruns the chosen search method rather than using D* or incremental A*. The mission has one robot, one parcel and a fixed task order. It does not optimize dispatch, deadlines, battery capacity or multi-robot coordination.

Practical applications

Setting	Potential use
Campus deliveries	Plan delivery routes between a depot, library and laboratories while avoiding closed or crowded walkways.
Warehouse logistics	Model a robot carrying parcels between storage and packing stations, then reroute when an aisle is blocked.
Hospital internal logistics	Explore routes for transporting supplies between departments while accounting for restricted or busy corridors.

These are potential applications of the route-planning approach. The current implementation remains a grid simulation; it does not connect to real robots, building maps or operational dispatch systems.

Future scope and responsible use

Extensions could add a heap-based frontier, incremental replanning, measured sensor calibration, time-dependent costs and multi-robot conflict handling. A physical deployment would also require localization, obstacle detection, human-aware navigation and controlled safety evaluation. The simulation stores no personal records and uses no camera input. Its illustrative probabilities should not be used to justify real navigation decisions.

8. Conclusion

The case study delivers a working simulation in which search, uncertain reasoning and a simple mission plan interact. Its strongest result is directly observable: A* can choose more moves yet achieve lower travel cost, while the agent can replace a disrupted route and report an impossible task honestly. The system provides a compact, reproducible demonstration of classical AI that can be explained through its states, costs, rules and measured outputs.

References

[1] UC Berkeley CS 188. 1.4 Informed Search.
inst.eecs.berkeley.edu/~cs188/textbook/search/informed.html
[2] UC Berkeley CS 188. 1.3 Uninformed Search.
inst.eecs.berkeley.edu/~cs188/textbook/search/uninformed.html
[3] UC Berkeley CS 188. 6.2 Probability Inference.
inst.eecs.berkeley.edu/~cs188/textbook/bayes-nets/inference.html